# Client-Side ANN on the Web: A Wikipedia Search Demonstration

David Denisov
University of Haifa
Haifa, Israel

**Live search demo:** https://wiki-ann-demo-20261022.pages.dev/client_wiki/
**Clustering lab:** https://wiki-ann-demo-20261022.pages.dev/client_wiki/benchmark.html
**Code and benchmarks:** https://github.com/DavidDenisov/ANN-on-the-Web

## Abstract

We demonstrate browser-based approximate nearest neighbour (ANN) search over 35.2 million English Wikipedia passages hosted on a static CDN. The browser embeds questions locally, uses inverted file (IVF) routing to select vector clusters for parallel download, ranks their vectors, and fetches text for the top results. This avoids a full index download or an application search server. Visitors submit questions and inspect passages, timings, and download costs; a companion lab measures the recovery–bandwidth trade-off. On a desktop with primed CDN caches, at 32k clusters, SQ6 centroids, and four probes, separate vector, ten-passage text and embedding means sum to an estimated 166–191 ms, depending on input length and excluding startup. Worker startup averages 2.46 s. At the same settings, adding a random noise vector $v$ with $\|v\|_2 = 0.4$ to each unit source vector, then renormalising, gives 98.1% source recovery@1; this is not real-question recall. Queries and embeddings stay local; requests may reveal access patterns.

## CCS Concepts

• **Information systems** → **Similarity search**.

## Keywords

approximate nearest neighbors, semantic search, client-side retrieval

**ACM Reference Format:**
David Denisov. 2027. Client-Side ANN on the Web: A Wikipedia Search Demonstration. In *Companion Proceedings of the ACM Web Conference 2027 (WWW '27).* ACM, New York, NY, USA, 4 pages.

## 1 Introduction

Semantic search on a static document collection usually requires a search service or a large local index. We demonstrate browser ANN over 35.2 million passages without either: local inverted file (IVF) routing [5] selects remote cluster files for parallel download. The probe count controls the number of clusters fetched and ranked.

We demonstrate a static-Web deployment of established ANN techniques: search exposes per-question costs, while the lab isolates routing and centroid compression. FP16 vectors occupy about 27 GB per index layout; each query fetches selected clusters and result text from Cloudflare Pages.

### 1.1 Related work

Our system builds on the author's earlier public image-search prototype, which is not demonstrated here because of its adult content. Related software includes the subsequently released ANNPack, which retrieves selected IVF lists through HTTP range requests.

SPANN [1] uses centroids in RAM to select posting lists on disk. Tiptoe [4, Sections 3–5] encodes its query and chosen cluster in an encrypted request. Its server computes encrypted similarity scores, which the client decrypts before privately retrieving result URLs. We download selected clusters as static files and rank plaintext vectors locally, exposing file requests to the host.

DiskANN [9] traverses a graph on SSD, using compressed vectors in RAM to guide neighbour reads. MeMemo [10] and WebANNS [6] keep HNSW indexes in browser RAM and IndexedDB; WebANNS evaluates up to 480,000 passages. Importing our remote corpus into such local indexes would require a large initial data transfer; we avoid that import by fetching only selected clusters in parallel.

More broadly, Pagefind (keyword search) and sql.js-httpvfs (SQL) also process selectively downloaded static data in the browser.

## 2 Demonstration scenario

The two public pages use the same Wikipedia passage collection.

*Wikipedia search.* Visitors enter questions or text and receive ranked passages linked to Wikipedia. Defaults are 32k clusters, SQ6 centroids, four probes, and ten results. Controls offer three cluster counts, six centroid formats, and 1–100 probes. The page reports embedding, centroid-loading, retrieval and text-loading times, and vector downloads. The presenter compares one and four probes for the example question (Figure 1), examining changed passages and bytes. Visitors then try their questions; examples illustrate behaviour, not relevance scores. We keep the walkthrough to one or four probes to limit downloads; queries can be cancelled.

*Clustering lab.* The lab (Figure 2) offers noise presets and checks for cluster count, probes and centroid format. Visitors select 1–10,000 precomputed perturbed-passage queries per configuration (default 10) and can cancel runs. Short runs uniformly sample a shared subset and illustrate behaviour; paper measurements use all 10,000 queries, excluding embedding. Probe and compression checks use 16k clusters; the paper adds a 32k supplement. Full checks can take hours and transfer hundreds of GB. At the poster, visitors try short checks and interpret retained tables and plots.

*Public access.* The presenter will preload the model and centroids on a Chromium laptop with Internet access and WebAssembly. A poster QR code supports visitor access; mobile performance is unvalidated. For slow networks or incompatible browsers, saved screenshots and retained plots support a narrated walkthrough. The supplied source includes the client, builders, figures, compact measurements, and self-hosting instructions.

*WWW '27, Dublin, Ireland*
2027.

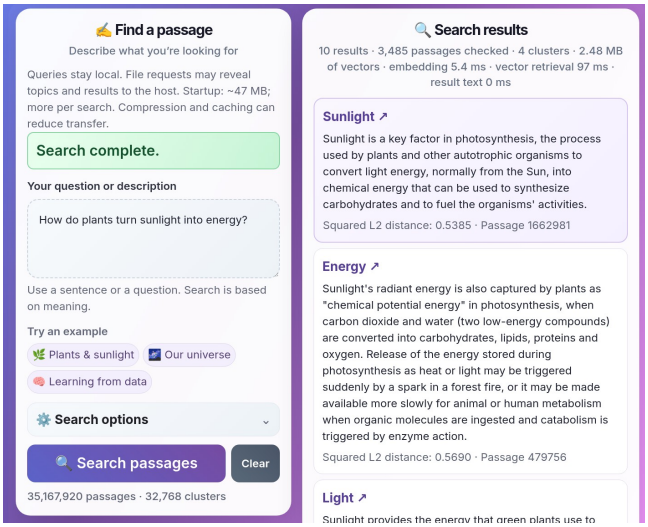



**Figure 1: Search with concise privacy/download notices and passages. Capture timings are illustrative; Section 4.5 estimates costs from component means. Text enlarged for clarity.**

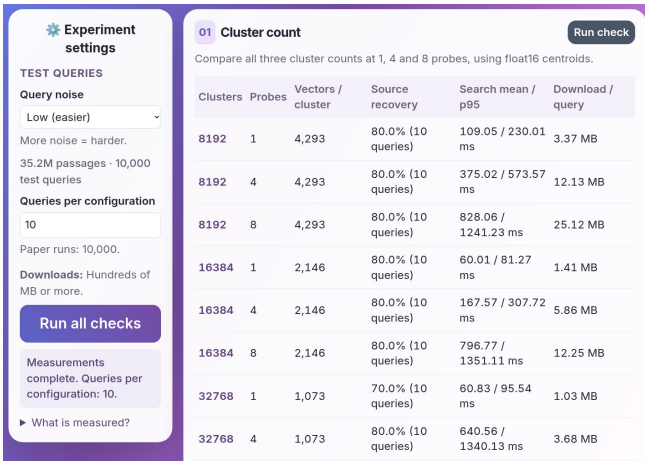



**Figure 2: Lab after a short cluster-count check: settings left, results right. Text enlarged for clarity.**

## 3 Method outline

Figure 3 traces local computation and the static files downloaded at startup and during search.

The Maloyan dataset [7] provides the passages and their 384-dimensional `all-MiniLM-L6-v2` embeddings [8].

For queries, Transformers.js [3] loads the Xenova ONNX conversion [13] in `q8` format, truncating inputs to 256 tokens.

*Index preparation.* We train $C$ centroids with Faiss $k$-means [2], using 20 iterations on about 2.1 million uniformly sampled passages. We assign each passage to its nearest centroid and store its ID and FP16 vector in that centroid's gzipped file. Centroids are published in FP16 and compressed formats; SQ6 uses six-bit scalar quantisation per component. Passage vectors remain FP16, occupying about 27 GB per index layout before compression.

*Result text.* Brotli files hold 500 contiguous passage IDs each, except the last. These text batches are shared across cluster counts.

*Static hosting.* The 70,336 text files use 71 branch deployments; vectors and centroids use 59 more in a second Pages project. Manifests map files to URLs. Our 1,000-file upload batches are a packaging choice, below the provider's 20,000-file Free-plan ceiling.

## 4 Supporting evaluation

Each index covers 35,167,920 passages. Tests use synthetic perturbations for routing and groups of ten passages for text retrieval.

*Query setup.* Configurations share 10,000 uniformly sampled distinct source passages. We draw a Gaussian vector $v \in \mathbb{R}^{384}$, rescale it to noise magnitude $\|v\|_2 = \alpha$, add it to the unit source vector, then renormalise. Routing tests use High ($\alpha = 0.4$); the lab also offers Low (0.05) and Medium (0.15).

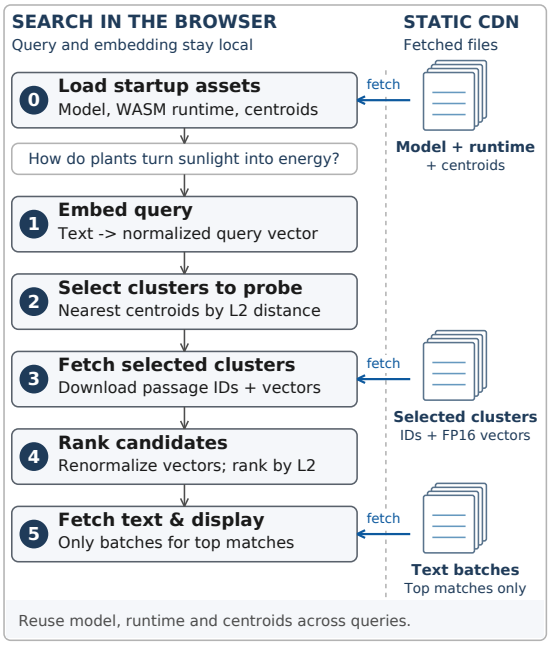



**Figure 3: Local query embedding and ranking, with model, runtime and centroid downloads at startup and selective vector and text downloads per query.**

*Perturbation scale and scope.* On 1,000 sources, mean query-source L2 displacement is 0.378, or 55.5% of the mean nearest-neighbour distance (0.681) from an exact full-corpus scan excluding self matches. This probes IVF boundaries; isotropic noise does not model question–passage distributions.

*Metric.* **Source recovery@1** measures how often a perturbed query ranks its source first. Exact search can favour another passage, so recovery is not ANN recall. Exact neighbours for these noisy queries, real-question relevance/recall@10, and q8 question-encoder effects are unmeasured. For SQ6/four probes, the 95% Wilson interval [12] is [97.77, 98.31]% (9,806/10,000).

*Test environment.* Experiments use Ubuntu 22.04 on an Intel Core i5-12400F PC with 32 GB RAM and a GTX 1660 SUPER GPU. GPU Faiss prepared the indexes. Browser experiments use Chromium 155; inference uses Transformers.js 4.3.1 with CPU-only WebAssembly and four threads. Separate FAST.com checks reported about 300 Mbit/s download; CDN throughput was uncontrolled.

*Embedding compatibility.* Re-embedding 1,000 uniformly sampled passages with the q8 browser encoder yields mean and 5th-percentile cosine similarities of 0.9924 and 0.9892 to the stored vectors; this passage check does not validate question retrieval.

FP16 probe sweeps use 2–5 probes at 8k and 2, 3, 4, 6, 8, 10 at 16k/32k; the 8k four-probe point reuses the original measurement. At 32k and four probes, we compare FP16, SQ8/SQ6/SQ4, symmetric INT8 [2], and PQ4 with 4-bit codes per eight-component block [5]. Cluster assignments and FP16 passage vectors stay fixed. Shared configurations run once.

*Hosting and timing.* Vector and text files are prefetched to prime CDN caches. CDN residency and connections remain uncontrolled; browser-cache bypass does not establish a cold CDN. Timed routing searches return one passage, fetch up to ten clusters in parallel and bypass browser HTTP caches. Untimed accuracy checks allow cached clusters. Table 1 reports recovery, timing and payload on the same timed queries.

Retrieval time includes routing, vector download, decompression, decoding and ranking; Sections 4.3–4.5 report text retrieval, startup and total query costs. Centroids load once per configuration; separate download/decode benchmarks use 1,000 trials per file. Tables give mean and 95th-percentile (p95) times, and gzipped vector payload in decimal MB ($10^6$ bytes). Configuration order was randomised; the original seed and supplement order are recorded.

## 4.1 Vector retrieval

With FP16 centroids, four probes recover over 98% of source passages at all three cluster counts (Table 1). More clusters reduce vector downloads and latency but increase centroid loading.

**Table 1: FP16 four-probe retrieval at $\alpha = 0.4$: source recovery@1, mean / p95 time (ms), and vector payload per query.** 10,000 **queries per setting.**

| Clusters | Probes | Recovery (%) | Mean/p95 | MB/query |
|---|---|---|---|---|
| 8192 | 4 | 98.4 | 462 / 662 | 13.65 |
| 16384 | 4 | 98.2 | 215 / 326 | 7.11 |
| 32768 | 4 | 98.1 | 135 / 182 | 3.98 |

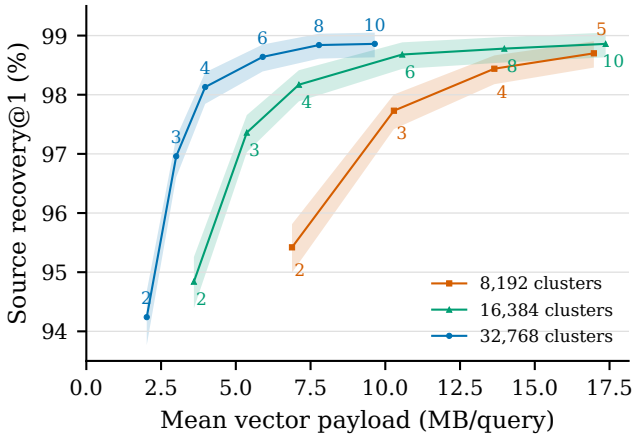



**Figure 4: Recovery versus vector payload (FP16, $\alpha = 0.4$; 10,000 queries/setting). Shading joins pointwise 95% Wilson intervals; labels mark probes.**

At 8k, two to five probes transfer 6.88–16.98 MB/query with 95.4–98.7% recovery. Figure 4 reports gzipped vector payload, excluding startup and text, because file bytes are unaffected by network conditions during multi-day runs. Observed retrieval time scaled approximately linearly with payload over this range.

## 4.2 Centroid loading costs

At 32k/four probes (Table 2), **SQ6 cuts centroid bytes by** 61.4%, from 23.24 to 8.98 MB. Mean loading in our runs falls from 929 ms (FP16) to 324 ms (SQ6); both round to 98.1% source recovery.

**Table 2: Centroid formats at 32k clusters and four probes: source recovery@1 on** 10,000 **queries ($\alpha = 0.4$), file size, and loading mean / p95 over 1,000 trials per file. Decode includes gzip decompression.**

| Format | Recovery (%) | Size (MB) | Fetch (ms) | Decode (ms) |
|---|---|---|---|---|
| FP16 | 98.1 | 23.24 | 675 / 692 | 254 / 260 |
| SQ8 | 98.2 | 11.23 | 300 / 339 | 113 / 115 |
| SQ6 | 98.1 | 8.98 | 224 / 261 | 100 / 102 |
| SQ4 | 97.4 | 4.86 | 83 / 102 | 88 / 91 |
| INT8 | 98.1 | 10.99 | 290 / 342 | 103 / 106 |
| PQ4 | 68.0 | 0.81 | 23 / 33 | 30 / 31 |

FP16, SQ8, SQ6, and INT8 differ by at most 0.09 percentage points; paired significance was not assessed. SQ4 and PQ4 reduce recovery at the tested code sizes. We choose 32k/SQ6 and four probes from this synthetic grid; higher cluster counts are untested.

### 4.3 Text retrieval

We measure 1,000 groups of ten passages, using 10,000 uniformly sampled distinct passage IDs. Text retrieval takes 34.9/54.9 ms (mean/p95), covering parallel batch downloads, Brotli decompression and text decoding. We prefetch files and bypass browser and decoded-text caches. DOM rendering is excluded.

### 4.4 Client setup

Startup spans worker creation to model and 32k SQ6 centroid readiness, excluding navigation. Across 100 fresh browser contexts in one Chromium process, it takes 2.46/2.53 s (mean/p95). Model (23.7 MB), WebAssembly runtime (14.3 MB) and compressed centroids (9.0 MB) total about 47 MB before HTTP compression. Loads overlap and bypass browser caching.

### 4.5 Total query cost

At 32k/SQ6 and four probes, vector retrieval averages 122.4 ms and ten-passage text loading 34.9 ms. Embedding averages 9.1, 16.9 and 33.8 ms for 64, 128 and 256 tokens, giving estimated totals of 166.4, 174.1 and 191.1 ms, respectively. Estimates combine separate runs, excluding startup, rendering, messaging and extra ranking.

### 4.6 Trade-offs and limitations

SQ8/PQ passage codes with optional rescoring could reduce vector transfer, but their ranking quality is untested. A server returning text alone is also an unmeasured baseline.

At 32k, cluster sizes have median 1,023, p95 2,188, and maximum 8,263 passages. The four largest gzip files sum to 19.32 MB: a four-probe upper bound, not a query percentile; byte tails are unmeasured. Varied real-question end-to-end mean/p95/max, cold-connection/CDN trials and throttled/mobile profiles are needed before generalising desktop timings.

*Privacy threat model.* Assuming trusted client code and dependencies, query text and embeddings stay local. Model and runtime assets load from Hugging Face and jsDelivr. The host/CDN may link IP addresses and timing to four of 32,768 cluster IDs and text-batch IDs. For uncached requests, intersecting these batches with downloaded candidate IDs may identify returned passages and help reconstruct the displayed result set; inference effectiveness is unmeasured. Displayed results do not establish which passages were read. Shared ID-based text batches support all three cluster counts. Co-location would avoid result-specific text requests but require one text layout per count, while selected clusters remain visible. PIR, as in Tiptoe [4], requires a different serving protocol.

## 5 Conclusion

Local IVF routing enables browser search over 35.2 million statically hosted passages. The demo exposes source recovery, latency, and bandwidth trade-offs; real-question quality, cold-CDN latency and mobile performance remain unevaluated.

## 6 Ethical use of data

We downloaded Maloyan's precomputed embeddings of 35,167,920 English Wikipedia passages (wikipedia-22-12-en) [7]. They were generated by applying the MiniLM encoder [8] to these passages. The encoder and its Xenova ONNX conversion [13] for browser queries are Apache-2.0 licensed. Result links credit Wikipedia contributors through source articles [11]. The supplied client and text exporters include a CC BY-SA 3.0 notice and `DATA-NOTICE.txt`, recording attribution and extraction/packaging changes. The embedding dataset specifies no separate licence. Archived passages may be outdated or biased, and linked articles may change or disappear. No human participants were involved.

## References


[1] Qi Chen, Bing Zhao, Haidong Wang, Mingqin Li, Chuanjie Liu, Zengzhong Li, Mao Yang, and Jingdong Wang. 2021. SPANN: Highly-efficient Billion-scale Approximate Nearest Neighborhood Search. In *Advances in Neural Information Processing Systems*, M. Ranzato, A. Beygelzimer, Y. Dauphin, P.S. Liang, and J. Wortman Vaughan (Eds.), Vol. 34. Curran Associates, Inc., 5199–5212. https://proceedings.neurips.cc/paper_files/paper/2021/file/299dc35e747eb77177d9cea10a802da2-Paper.pdf

[2] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2026. The Faiss Library. *IEEE Transactions on Big Data* 12, 2 (2026), 346–361. doi:10.1109/TBDATA.2025.3618474

[3] Hugging Face. 2024. Transformers.js: State-of-the-art Machine Learning for the Web. GitHub repository. https://github.com/huggingface/transformers.js Accessed October 24, 2026.

[4] Alexandra Henzinger, Emma Dauterman, Henry Corrigan-Gibbs, and Nickolai Zeldovich. 2023. Private Web Search with Tiptoe. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*. ACM, 396–416. doi:10.1145/3600006.3613134

[5] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33, 1 (2011), 117–128. doi:10.1109/TPAMI.2010.57

[6] Mugeng Liu, Siqi Zhong, Qi Yang, Yudong Han, Xuanzhe Liu, and Yun Ma. 2025. WebANNS: Fast and Efficient Approximate Nearest Neighbor Search in Web Browsers. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '25)*. Association for Computing Machinery, New York, NY, USA, 2483–2492. doi:10.1145/3726302.3730115

[7] Narek Maloyan. 2023. Wikipedia-22-12-en embeddings with all-MiniLM-L6-v2. Hugging Face dataset. https://huggingface.co/datasets/maloyan/wikipedia-22-12-en-embeddings-all-MiniLM-L6-v2 Accessed October 24, 2026.

[8] sentence-transformers. n.d. all-MiniLM-L6-v2. Hugging Face model card. https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2 Accessed October 30, 2026.

[9] Suhas Jayaram Subramanya, Devvrit, Rohan Kadekodi, Ravishankar Krishnaswamy, and Harsha Vardhan Simhadri. 2019. DiskANN: fast accurate billion-point nearest neighbor search on a single node. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[10] Zijie J. Wang and Duen Horng Chau. 2024. MeMemo: On-device Retrieval Augmentation for Private and Personalized Text Generation. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '24)*. Association for Computing Machinery, New York, NY, USA, 2765–2770. doi:10.1145/3626772.3657662

[11] Wikipedia contributors. n.d. Wikipedia:Copyrights. https://en.wikipedia.org/wiki/Wikipedia:Copyrights Accessed October 30, 2026.

[12] Edwin B. Wilson. 1927. Probable Inference, the Law of Succession, and Statistical Inference. *J. Amer. Statist. Assoc.* 22, 158 (June 1927), 209–212. doi:10.1080/01621459.1927.10502953

[13] Xenova. n.d. Xenova/all-MiniLM-L6-v2. Hugging Face model card. https://huggingface.co/Xenova/all-MiniLM-L6-v2 Accessed October 30, 2026.